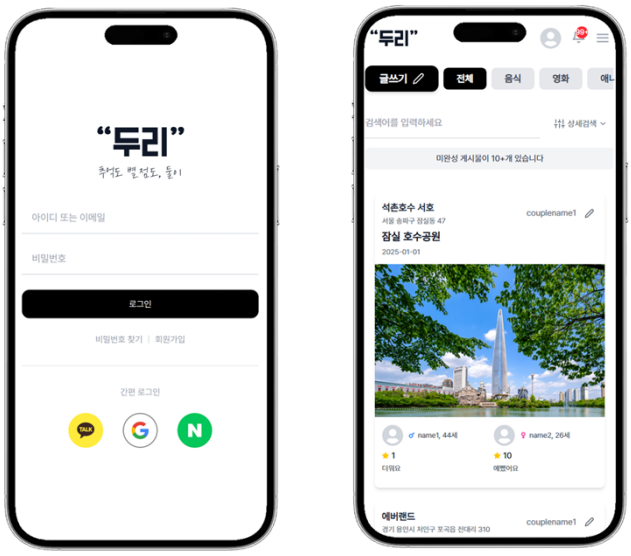
# 두리

2025. 05 – present

Github



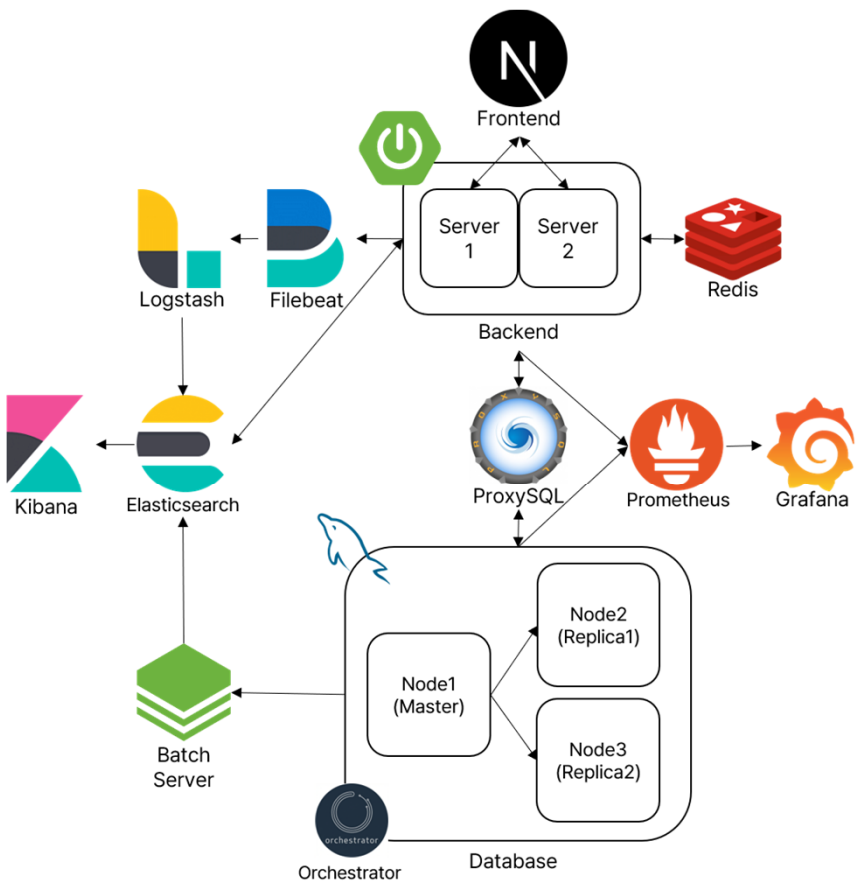

## 개요

커플의 데이트 경험에 대한 리뷰를 남길 수 있는 앱입니다. 데이트 장소, 음식, 영화, 등 데이트 관련 리뷰를 추억으로 남길 수 있습니다. 커뮤니티 기능을 사용해서 다른 커플들의 데이트 리뷰, 인기있는 데이트 장소 등을 확인할 수 있습니다.

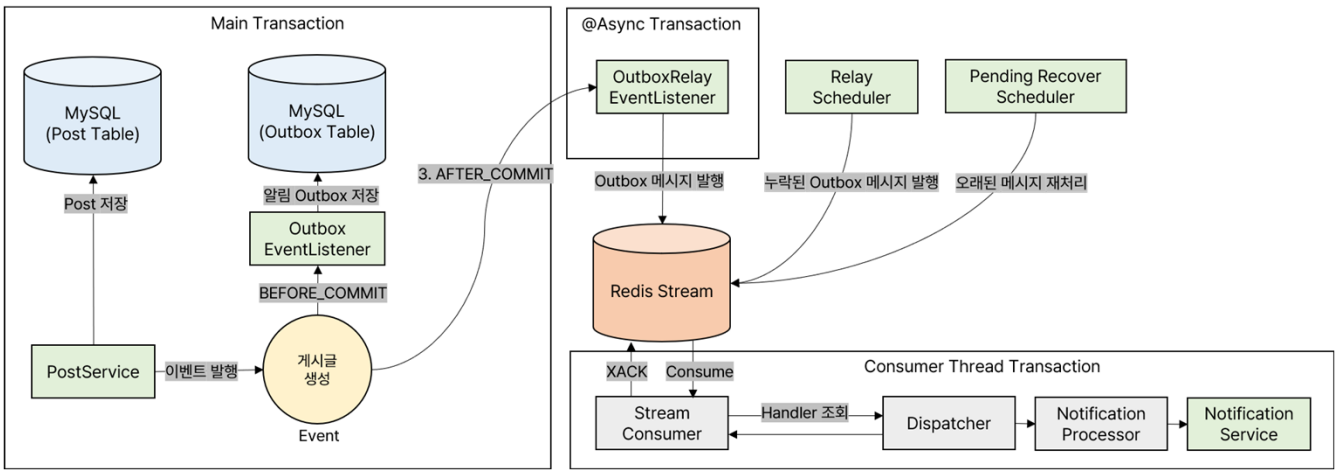
## 사용 기술

- Java21, Spring Boot 3.4.5, Spring MVC, Spring Data JPA, Spring Security, Spring Batch 5, JWT, OAuth2.0, QueryDsl, Spring REST Docs
- MySQL, Redis, Elasticsearch
- Next.js
- AWS (EC2, ELB, RDS, S3), Git, Github Actions, Docker, ProxySQL, Orchestrator, Prometheus, Grafana, Logstash, Kibana

## 프로젝트 아키텍쳐



# 이벤트 기반 아키텍처 개선 (Outbox 패턴, Redis Stream)




## 문제 (Problem)

Spring Application Event + @TransactionalEventListener(AFTER_COMMIT) 기반으로 게시글 생성 시 통계 증가, 알림 전송, 검색 인덱싱 등의 부가 기능을 분리.
이를 통해 관심사 분리는 달성했지만, 다음과 같은 문제가 있다.

### [커넥션 점유 문제]

- 부가 기능을 메인 트랜잭션과 분리하기 위해 REQUIRES_NEW를 사용했지만,이 방식은 스레드는 동일하고 커넥션만 추가로 점유하는 구조.
  -> 하나의 요청이 동시에 2개의 DB 커넥션을 점유하게 됨
- 부가 로직이 지연될 경우 메인 트랜잭션 커넥션도 함께 점유 상태 유지
- 트래픽 증가 시 커넥션 풀 고갈 가능성 증가
- 동시 요청 증가 시 Deadlock 발생 가능

### [데이터 유실 방지 불가]

- @Async + AFTER_COMMIT 구조로 트랙잭션 분리 문제는 개선 가능하지만 서버 장애 발생으로 비동기 스레드가 이벤트 처리에 실패하면 이벤트가 유실됨.
  -> 처리 실패에 대한 재시도 로직 부재

## 해결 (Solution)

### [Outbox 패턴 도입]

- Outbox 패턴은 데이터 저장과 이벤트 기록을 하나의 트랜잭션으로 묶어 원자성을 보장
  -> 이벤트 유실 방지 (DB에 기록되므로)
  -> 실패 시 재처리 가능
  -> 비동기 처리와 정합성 동시 확보

설계

1. BEFORE_COMMIT 단계에서 게시글 저장과 함께 Outbox 데이터를 동일 트랜잭션에서 저장
2. AFTER_COMMIT + @Async로 메인 트랜잭션 커밋 이후 Outbox 데이터를 조회하여 Redis Stream으로 발행
3. Redis Stream 기반 메시지 처리 (Consumer)
4. 재시도 및 복구 전략
   - 즉시 발행 실패 -> 스케줄러 재발행
   - Consumer 장애 -> Pending 메시지 재처리 (XCLAIM)
   - 반복 실패 -> retryCount 기반 FAILED 처리
5. 확장성 있는 구조
   - Dispatcher + 전략 패턴 -> 신규 이벤트 추가 시 기존 코드 수정 없이 확장 가능

## 결과 (Result)

**[정합성과 안정성 확보]**

- 이벤트 유실 가능성 제거 (DB 기반 Outbox 저장)
- 장애 발생 시에도 재처리 가능한 구조 확보

**[성능 및 사용자 경험 개선]**

- 부가 기능을 비동기 처리로 분리하여 요청 응답 시간과 완전히 독립
- 알림/통계 실패가 메인 트랜잭션에 영향을 주지 않도록 개선

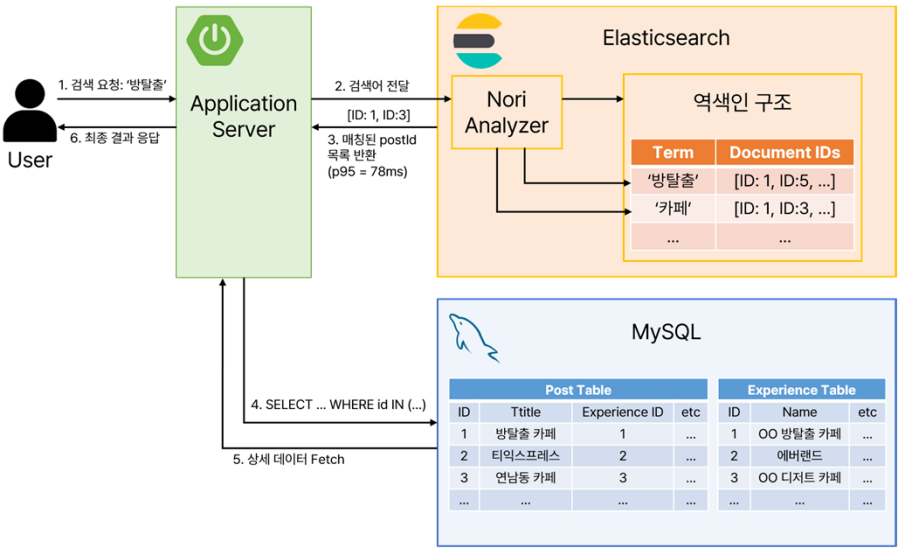
**[구조적 확장성 확보]**

- 새로운 부가 기능 추가 시 핵심 서비스 수정 없이 확장 가능
- 이벤트 처리 로직이 Dispatcher 기반으로 분리되어 유지보수성 향상

**[운영 관점 개선]**

- retryCount 기반 실패 이벤트 추적 및 모니터링 가능
- 커스텀 메트릭을 통한 장애 탐지

# 검색 아키텍처 개선 (LIKE -> Full-Text Search -> Elasticsearch)




## 문제 (Problem)

### [대용량 데이터 검색 성능 저하]

- QueryDSL의 containsIgnoreCase() 기반 검색(SQL: LIKE %keyword%) 사용 중
  -> 데이터 증가 시 검색 응답 시간이 최대 60,735ms 가량 소요되는 성능 저하 발생.

### [문제 분석]

- LIKE '%keyword%'는 문자열 앞이 고정되지 않아 B-Tree 인덱스 범위 탐색 불가
  -> Full Table Scan 발생.
- 검색, 정렬, 페이징을 모두 DB에서 처리하면서 데이터 증가 시 선형적으로 성능 저하.

## 해결 (Solution)

### [MySQL Full-Text Search 도입]

선택 이유

- 역 색인 기반 검색으로 후보 row를 빠르게 식별 가능.
- 별도의 외부 인프라 구축 없이 기존 자원을 활용하여 구현 가능.

구현

- FULLTEXT INDEX + N-gram Parser 적용, innodb_ft_min_token_size=2.
- JPA에서 MATCH ... AGAINST 사용을 위해 Hibernate FunctionContributor 확장해 Custom Dialect 구현.
- score 기반 정렬 적용.

결과

- 응답 시간 60,735ms -> p95 약 82ms 수준으로 개선.

### [Full-Text Search 한계 분석]

1. 정렬 병목

- MATCH...AGAINST 결과셋과 일반 B-Tree 인덱스 컬럼(created_at, rate) 간의 복합 인덱스 활용 불가.
  -> 검색 결과가 대량일 경우, 정렬 처리를 위한 Filesort 발생 하여 응답속도 증가.

2. 커서 기반 페이지네이션 비효율

- Infinite Scroll 구현시 WHERE 절의 커서 조건(post_id < ?)이 MATCH 결과 전체를 스캔 후 필터링 수행함.
  -> Offset 페이징 방식과 동일하게 페이지가 뒤로 갈수록 성능 저하.

### [최종 해결: Elasticsearch 도입]

1. Nori 형태소 분석기 적용

- 한글 형태소 기반 분석 -> N-gram의 과도한 매칭 문제 제거.

2. search_after 기반 커서 페이지네이션

- 정렬 기준: createdAt DESC, post_id DESC(중복 방지용 보조 정렬 필드).

3. Elasticsearch + DB 하이브리드 구조

- PostDocument은 검색용 필드만 index
- Elasticsearch -> postId 목록 검색, MySQL -> findByIdIn 조회 후 Entity 그대로 반환
  -> ES Document ↔ Entity 매핑 제거, JPA 연관관계 유지, 인덱스 크기 최소화.

## 결과 (Result)

| 방식 | p95 응답시간 |
|---|---|
| LIKE | 60,735ms |
| FTS | 82ms |
| Elasticsearch | 78ms |

- 응답 시간 60,735ms-> 78ms (약 99.8% 개선).
- 대용량 데이터에서도 응답 시간의 안정성 유지.
- 검색 전용 모델 분리로 구조적 확장성 확보

# 게시글 통계(좋아요/댓글/조회) 동시성 제어

## 문제 (Problem)

### [동시 요청 집중]

- 좋아요/댓글/조회수 등 통계 데이터는 특정 postId에 요청이 집중되는 특성이 있음
  -> SELECT -> Modify(+1) -> UPDATE 방식은 동시에 실행되면 Lost Update가 발생해 정합성 깨짐

### [문제 분석]

- 통계 업데이트는 빈번한 UPDATE이므로 인덱스(views, likes, ...) 적용 시 B-Tree 재정렬 발생
  -> DB 리소스(페이지 분할, Redo/Undo Log 증가) 비용이 커짐.
- 단일 서버에서는 synchronized로 직렬화 가능하지만, 다중 인스턴스에서는 서버 간 JVM Lock 공유 불가

## 해결 (Solution)

### [설계: PostStat 분리]

- 통계를 Post 엔티티에 포함하면 작은 카운트 변경에도 큰 row가 갱신됨.
  -> 락 범위 확대, Buffer Pool 비효율, Redo/Undo Log 오버헤드가 커짐.
- 카운터 전용(PostStat) 테이블로 분리해 업데이트 비용을 분리 및 축소

### [동시성 제어 후보 비교]

성능/정합성 테스트(k6 constant-arrival-rate, 30s)

- 테스트 환경: CPU Ryzen 5 3600 / RAM 16GB
- RPS 10, 50, 100, 150, 200, 300 측정

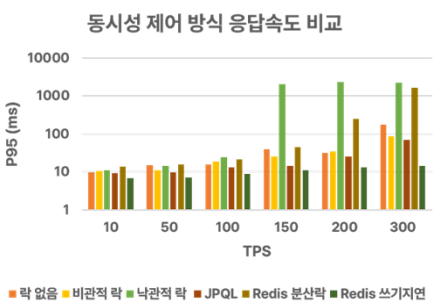
비교 대상(6종)

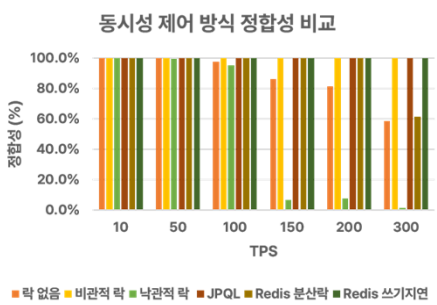
- No Lock
- 비관적 락
- 낙관적 락(+재시도)
- JPQL Atomic Update
- Redis 분산락
- Redis 쓰기지연

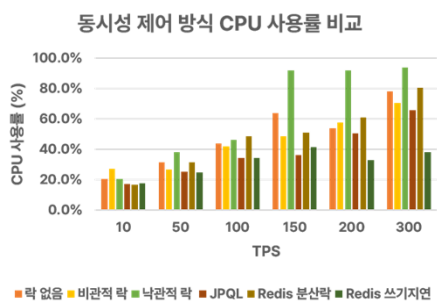
선택 기준

- 정합성(실제 DB 증가분 / 요청 수)
- p95 응답시간
- 실패율
- CPU 사용률
- 운영 복잡도(추가 인프라/튜닝/장애 대응)

[결과 분석]




| 제어 방식 | 특징 |
|---|---|
| No Lock | RPS 증가에 따라 정합성이 58.6%까지 감소하여 데이터 신뢰성 확보 불가. |
| 비관적 락 | 고부하 환경에서도 정합성 100%를 보장하나, 리소스 경합으로 인해 응답 시간이 선형적으로 증가함. |
| 낙관적 락 | 동일 Row 집중 접근 시 충돌 오버헤드로 인해 실패율이 98.5%에 달하며 고 트래픽 환경에 부적합. |
| JPQL | 별도 인프라 없이 P95 71ms, 정합성 100%를 달성하여 현 단계에서 가장 효율적인 비용 대비 성능을 보임. |
| 분산 락 | 락 획득 타임아웃 및 RTT 오버헤드로 인해 300 RPS 구간에서 실패율이 급증하며 병목 발생. |
| 쓰기지연 | P95 14ms의 가장 좋은 성능 및 DB 인덱스 갱신 부하 최소화가 가능하나, 운영 복잡도 및 유실 대응 설계 필요. |

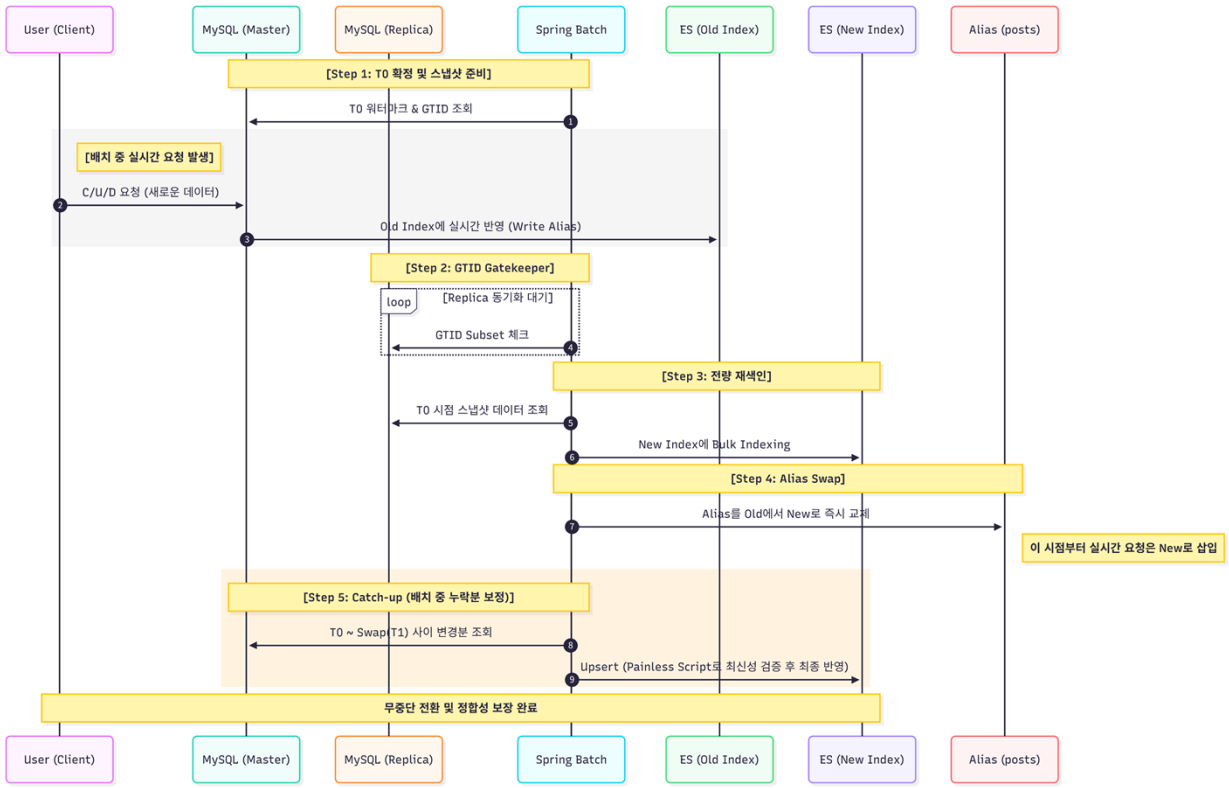
### [최종 선택: JPQL Atomic Update]

- 현재 트래픽 규모에서는 추가 인프라 없이 정합성 100%를 유지하면서, 응답시간과 CPU가 가장 안정적으로 유지되는 JPQL 방식 선택.
- Redis 쓰기 지연은 RPS 300에서 p95 14ms로 가장 빠르지만, 현재 단계에서는 운영 복잡도를 감안해 트래픽 증가로 DB 인덱스 갱신 비용이 병목이 되는 시점에 Redis 쓰기 지연으로 전환.

## 결과 (Result)

- 가상부하 RPS 300에서도 p95 71ms 수준으로 안정적 동작 및 정합성 100% 유지.

# Post 재색인 일일 배치 설계




## 문제 (Problem)

### [사용자/동의어 사전 최신화]

- 사용자/동의어 사전이 변경됐을 때 인덱스 데이터를 재색인 해야 변경분이 적용됨.

### [데이터 정합성 불일치]

- Post 검색은 Elasticsearch로 처리하고, 실데이터는 DB에 저장하는 구조에서 데이터 불일치가 발생
  -> 글은 있는데 검색이 안 되거나 삭제했는데 검색 결과에 남는 현상.

### [문제 분석]

- ES 반영이 실시간 이벤트 리스너(비동기) 기반이기 때문에 다음과 같은 상황에 누락 가능:
  -> DB 트랜잭션은 커밋됐지만 리스너 실행 실패
  -> 네트워크/ES 장애로 인덱싱 요청 유실

## 해결 (Solution)

### [재색인 배치 도입]

DB를 Source of Truth로 두고 새 인덱스 생성 -> 전량 재색인 -> alias swap 무중단 전환
-> 배치 실패 시에도 기존 인덱스로 검색 유지.

### [배치 중 변경 누락 방지]

- 배치 시작 시점 T0 = (modified_at, id) 워터마크 저장
- swap 직후 T1 워터마크 저장, (T0, T1] 구간 DB 변경분을 재조회하여 신규 인덱스에 보정
- 보정(Catch-up) 진행 시 실시간 리스너가 반영한 최신 문서를 덮어쓰는 문제
  -> Upsert 시 DB.modified_at > ES.modified_at 인 경우에만 업데이트하도록 Painless Script 적용

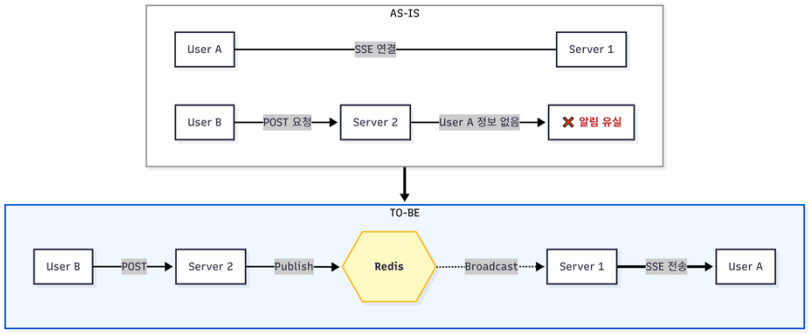
### [Replica 복제 지연 대응]

- Replica에서 전량 색인을 수행하면 복제 지연으로 스냅샷 누락 위험 존재
  -> GTID 기반 정합성 체크 후 스냅샷 수행
  -> Catch-up은 Master에서 처리해 복제 지연 리스크 제거

## 결과 (Result)

- 동의어/사용자 사전 변경시 배치 실행마다 변경분 적용
- 인기글 산출 등 부가 배치 작업에 동일 파이프라인 재사용 가능
- 데이터 정합성 문제가 발생해도 일 1회 배치가 DB 기준으로 자동 복구

## 분산 환경의 SSE 알림 유실 방지




### 문제 (Problem)

**[분산 환경에서 알림 유실]**
서버를 scale-out 한 이후 간헐적으로 실시간 알림이 누락됨.

**[문제 분석]**
다중 서버 환경에서 사용자의 SSE 세션이 특정 서버에 종속(Stateful)되어, 이벤트 발생 서버와 연결 서버가 불일치할 경우 알림이 유실된다.

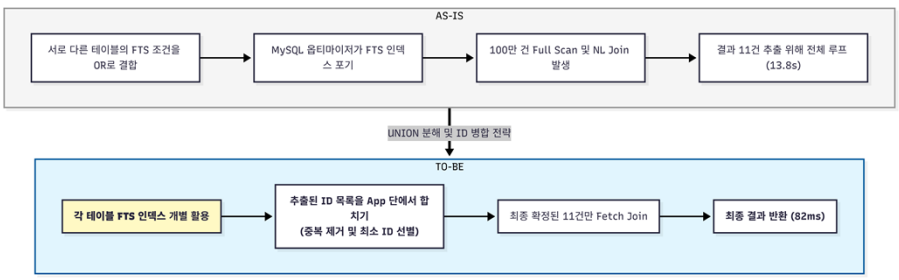
### 해결 (Solution)

**[Redis Pub/Sub 기반 메시지 브로커 도입]**
- 알림 발생 서버는 특정 사용자의 ID를 포함한 메시지를 Redis 채널에 Publish.
- 모든 서버가 해당 채널을 Subscribe하여 실시간으로 메시지를 전파.
- 각 서버는 Local 세션 저장소를 조회해서 해당 사용자와 연결된 세션이 존재할 경우에만 SSE 전송.

### 결과 (Result)

- 서버 scale-out에 제약이 없는 Stateless한 알림 아키텍처 확립
- 실시간 사용자 경험 개선

## FTS 검색 성능 개선 - OR 조건 인덱스 무력화




### 문제 (Problem)

**[검색 속도 저하]**
- FTS 검색 결과 11건 조회에 13.8초 소요

**[문제 분석]**
- 현재 쿼리 구조 match(post.title) OR match(experience.name)

EXPLAIN ANALYZE 확인 결과
- post 테이블 약 100만 건 index lookup
- experience 테이블과 Nested Loop Join (1e+6 loops)
- 마지막 단계에서 OR 조건 필터링 수행

-> MySQL은 OR 조건에서 FTS 인덱스를 선필터로 활용하지 못함
-> 대량 조인 이후 OR 필터링 구조로 실행되어 11건 조회 위해 100만건 조인/루프 발생

### 해결 (Solution)

**[OR 조건을 UNION 개념으로 분해]**
- OR는 인덱스 효율을 깨뜨리기 때문에 각 FTS 조건을 독립 쿼리로 실행
  -> 각 쿼리가 자기 FTS 인덱스를 100% 활용 가능
- 조인 후 필터링 구조를 선 필터링 후 조인 구조로 전환

실행 과정
1) match(post.title) 기준으로 ID 조회
2) match(experience.name) 기준으로 ID 조회
3) 애플리케이션 단에서 ID Set으로 병합(중복 제거)
4) 최종 ID 11건만 fetch join 수행

### 결과 (Result)

- 검색 응답 시간 13.8초 -> 82ms
- 데이터 100만 건 규모에서도 안정적 응답 유지

# JPA JOINED 상속(@Inheritance + @DiscriminatorColumn) 도입 후 조회 병목 해결

## 문제 (Problem)

[전체 게시글 조회 응답 속도 증가]

- @Inheritance(strategy = JOINED) + @DiscriminatorColumn로 Experience를 Place / Video 하위 테이블로 분리
- 기존 전체 게시글 조회(ORDER BY date DESC, LIMIT 11) 쿼리 응답에 88초 소요

[문제 분석]

```
| -> Limit: 11 row(s)  (actual time=88850..88850 rows=11 loops=1)
    -> Sort: p1_0.`date` DESC, p1_0.post_id DESC, limit input to 11 row(s) per chunk
(actual time=88850..88850 rows=11 loops=1)
        -> Stream results  (cost=1.19e+6 rows=969242) (actual time=0.309..87590
rows=1e+6 loops=1)
            ...
              -> Single-row index lookup on c1_0 using PRIMARY
(couple_id=p1_0.couple_id)  (cost=0.496 rows=1) (actual time=0.00221..0.00223 rows=1
loops=1e+6)
            ...
 |
```

- Experience를 fetch join하면서 하위 테이블(place, video)의 LEFT JOIN이 추가
  -> dtype에 상관없이 join 대상이 늘어나 옵티마이저가 실행 계획을 재선택
- Post를 전부 읽고 다른 테이블을 전부 조인한 뒤 정렬해서 11건만 남기는 실행계획이 선택됨
  -> 기존의 정렬 인덱스 + LIMIT 조기 종료(Nested Loop 기반) 계획이 Hash join으로 변경되어 LIMIT 11이 무력화 되면서 병목 발생 (rows=1e+6, loops=1e+6, 88초)

## 해결 (Solution)

[쿼리 구조 변경]

Step 1) 커버링 인덱스로 post_id만 11개 선 조회

- SELECT post.id만 수행
  -> 테이블 데이터 페이지 접근을 최소화하고, 인덱스 레벨에서 정렬+LIMIT 종료

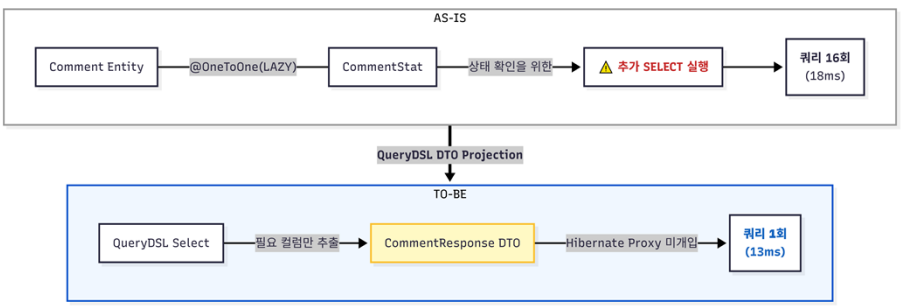
Step 2) 확보한 ID(11개)에 대해서만 fetch join으로 상세 조회

- WHERE post.in IN (ids)로 범위를 11건으로 고정
- 조인이 많아도 입력 자체가 11건
  -> 상속 JOINED로 place/video가 붙어도 loops=11 수준에서 종료

## 결과 (Result)

- 조회 응답 시간 88초 -> 0.31ms
- 정상 상태 대비 0.42ms -> 0.31ms (응답 속도 26% 추가 개선)

# N+1 문제 해결 – @OneToOne LAZY 프록시 한계 대응




## 문제 (Problem)

**[추가 쿼리 실행]**

- 대댓글 조회 API에서 Fetch Join을 적용했음에도, 쿼리 1회로 끝나야 할 요청이 총 16개의 쿼리 실행.

**[문제 분석]**

- Comment ↔ CommentStat가 nullable @OneToOne(mappedBy) 구조
- Hibernate는 null을 프록시로 감쌀 수 없음
  -> CommentStat 존재 여부 확인을 위해 추가 SELECT 실행
- 결과적으로 LAZY 설정임에도 EAGER처럼 동작

## 해결 (Solution)

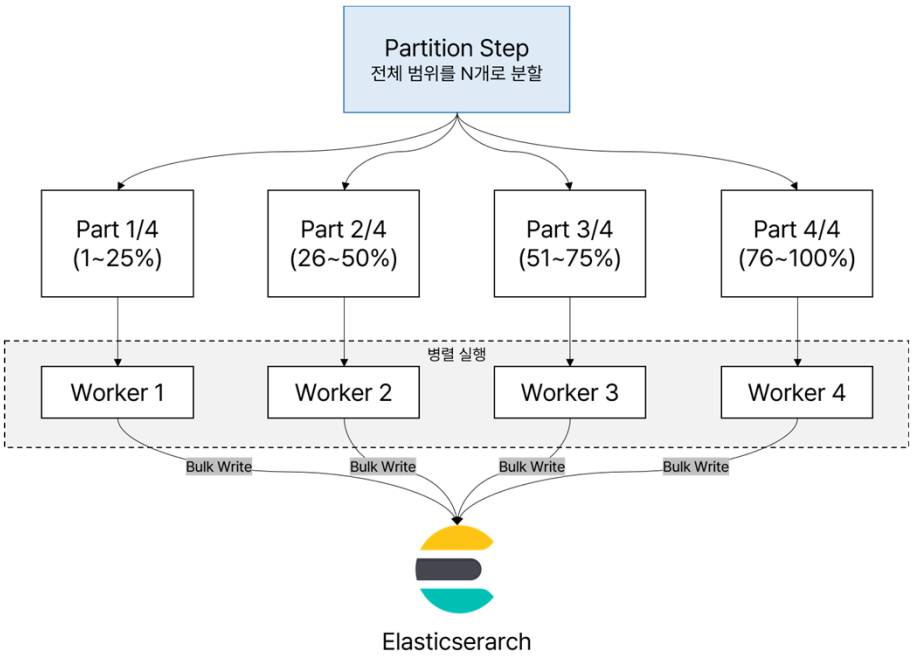
**[DTO Projection 전환]**

- 조회 전용 API이기 때문에 연관관계 탐색 불필요.
  -> 엔티티를 로딩하지 않고 필요한 컬럼만 DTO로 조회 (Hibernate 프록시 개입 제거)

## 결과 (Result)

- 대댓글 조회 API 쿼리 수 15개 감소 (16 -> 1)
- 조회 속도 28% 향상 (18ms -> 13ms)

# Post Reindex 일일 배치 성능 개선




## 문제 (Problem)

### [재색인 배치 실행 시간 지연]

- Elasticsearch 재색인 배치(약 200만 건)처리 시, 3시간 기준 가동률 30% 미만.

### [문제 분석]

EXPLAIN ANALYZE 확인 결과

- post 중심으로 7개 테이블 LEFT JOIN을 먼저 수행한 뒤
- 129만 건(rows=1.29e+6)을 조인/정렬해서
- 최종적으로 LIMIT 2000만 남기는 실행 계획이 선택됨 (쿼리 900초 소요)

-> 필요한 2,000건을 확정하기 위해 129만 건을 읽고 버리는 구조.

## 해결 (Solution)

### [지연 조인]

실행 단계

Step1) 커버링 인덱스로 post_id 2,000건만 선별

- 정렬+ LIMIT 2000을 인덱스 레벨에서 종료

Step2)확정된 ID에 대해서만 상세 테이블 조인

- 조인 횟수는 무조건 2,000회로 상한 고정
- dtype 조건으로 Place/Video 관련 테이블 조인을 분기하여 불필요 조인 감소

효과

- 쿼리 시간 900초 -> 1.46초
- 총 배치 시간 3시간+ -> 31분

### [지연 조인 한계 분석]

- 페이지당 1.46초 × 500회 ≒ 총 배치 시간 11분 이상 소요 예상, 실제 31분 소요

원인 분석

- 조인 단계에서 Single-row index lookup이 2000번 반복되어 디스크 랜덤 I/O가 누적됨
  -> 물리적 디스크 I/O 한계

### [2차 개선: Spring Batch Step Partitioning]

- modified_at 범위를 파티션으로 분할해 각 Worker Step이 독립적으로 read -> process -> write 수행

효과

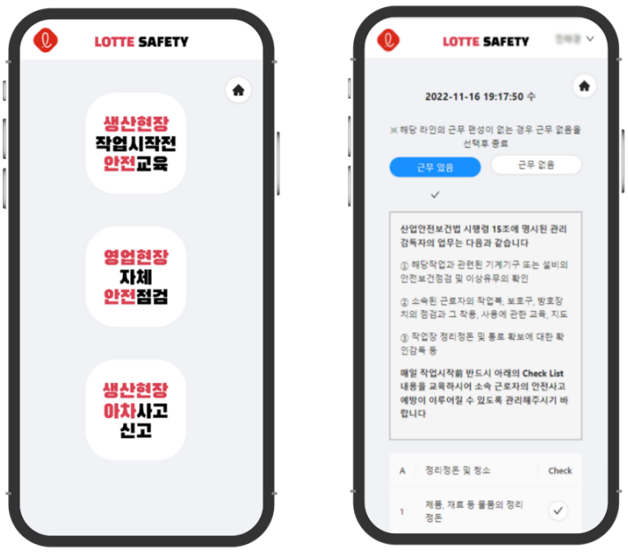
- 총 배치 시간: 31분 -> 8분 15초

## 결과 (Result)

- 200만 건 재색인 배치 처리 시간 3시간 20분+ (60만 건 처리 중 정체)
  -> 31분 (지연 조인 적용)
  -> 8분 15초 (Partitioning 적용)

# Lotte Saftey

2021. 05 – present



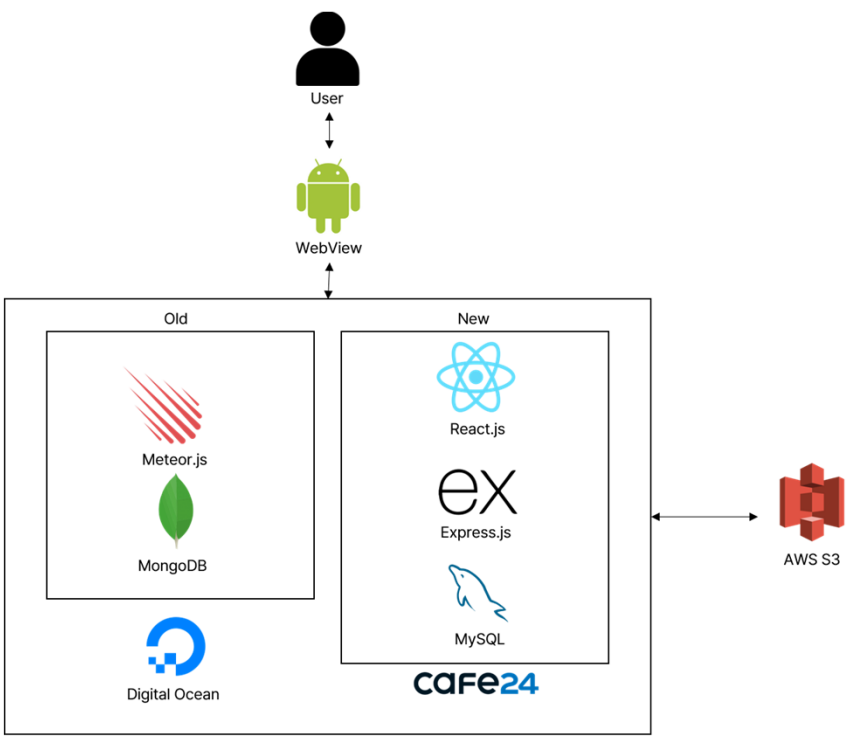

## 개요

롯데웰푸드 주식회사 안전경영팀 외주 프로젝트.
공장 생산현장 및 영업현장 안전관리 통합 모바일 앱.

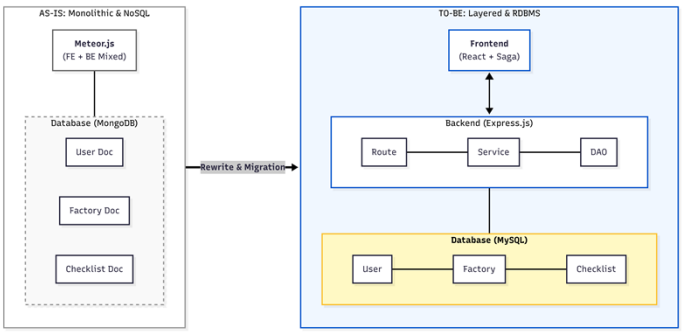
## 사용 기술

- Express.js, Meteor.js(Legacy)
- MySQL, MongoDB(Legacy)
- React.js
- Android Studio, AWS S3, Git, Cafe24, DigitalOcean(Legacy)

## 프로젝트 아키텍쳐



## 프로젝트 아키텍쳐 재구성




## 문제 (Problem)

**[Meteor.js, MongoDB 선택의 구조적 한계]**
- 프론트엔드와 백엔드 로직이 결합되어 유지보수 시 영향 범위 파악이 어려움
- User-Factory-Worksite-Checklist 등 관계 중심 정형 데이터에 부적합
  -> 참조 무결성 미보장으로 Orphaned Data 발생

## 해결 (Solution)

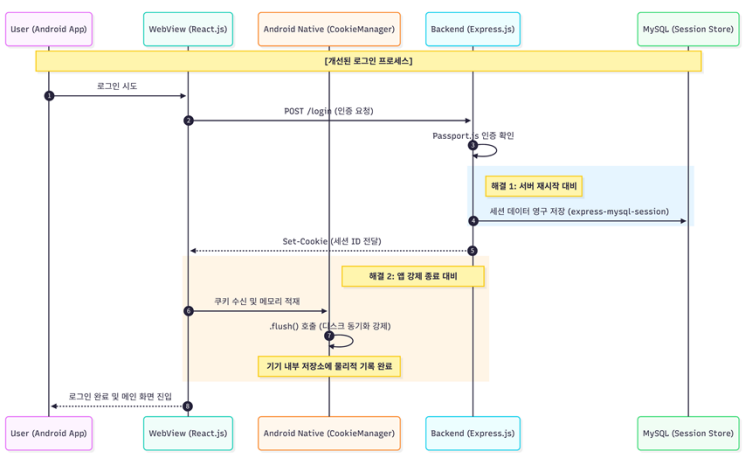
**[아키텍처 재구성]**
- Back-end: Express.js 기반 Layered Architecture로 책임 분리
- Front-end: React + Redux-Saga 도입으로 비동기 로직과 상태 관리 분리

**[데이터베이스 전환]**
- MongoDB -> MySQL
- Foreign Key 기반 관계 설계로 참조 무결성 강제
- ON DELETE CASCADE, 인덱싱을 통한 정합성·성능 동시 확보

## 결과 (Result)

- 유지보수시 코드 분석 시간 감소
- ACID 트랜잭션 기반 처리로 데이터 불일치 제거
  -> 운영 환경에서 안정적인 데이터 일관성 유지

## 로그인 유지 기능이 풀리는 문제 해결




## 문제 (Problem)

**[로그인 유지 기능 비정상 작동]**
WebView 앱 자동 로그인 (로그인 유지) 환경에서 잦은 로그아웃 발생

**[문제 분석]**
- 백엔드 서버에서 MemoryStore 기반 세션 사용
  -> 서버 재시작 및 배포 시 모든 세션 소실
- Android 앱 프로세스 종료 시 쿠키가 디스크에 기록되지 않음
  -> WebView 쿠키가 유실되어 로그인 정보 소실

## 해결 (Solution)

**[세션 영속화]**
- Express.js 세션 저장소를 MySQL로 이전
- express-mysql-session 적용서버 재시작과 무관하게 로그인 상태 복구 가능

**[WebView 쿠키 안정화]**
- 로그인 성공 시 CookieManager.flush() 호출
- 메모리 쿠키를 즉시 기기 저장소에 기록하도록 보강

## 결과 (Result)

- 서버 재시작, 앱 강제 종료 상황에서도 자동 로그인 유지되어 UX 향상.